Azure SQL specific audit actions

Execution guide | Prepared by Petar Trbos

Please use the enclosed files to demonstrate three scenarios in your own approved demo environment: table creation/deletion, DELETE versus TRUNCATE, and database-user changes. Follow the four steps in order and confirm the matching records in your Log Analytics workspace.

Scope Use a disposable Azure SQL logical server and user database, not production. The expanded server audit policy covers ALL databases on that logical server and increases audit volume. Do not change network policy, grant roles or weaken authentication solely to complete this exercise.

Before you begin

1. Have an existing Azure SQL logical server/user database and a Log Analytics workspace. Enable server auditing to Log Analytics ONLY in the portal: SQL server > Security > Auditing > On > Log Analytics > your workspace > Save. Keep Storage and Event Hub off for this exercise.

2. Confirm the correct tenant/subscription, SQL access as the disposable database administrator, and permission to manage server audit settings/read diagnostic settings and query the workspace. Azure Contributor does not itself grant SQL database access.

3. Verify approved network connectivity. If NSP is enforced, the SQL source and workspace must share the appropriate perimeter; enable the SQL server's managed identity as required by your design. Follow your organization's controls rather than copying another subscription's firewall configuration.

4. Unzip this package. It contains this guide in Word/PDF and the four numbered script files. Use the filenames below; no separate startup or test-results document is required.

File	Use
01-Expand-Audit.ps1	Step 1: expand the existing server audit policy.
02-Verify-Expanded-Audit.ps1	Step 2: verify groups and the master diagnostic route.
03-Execute-Specific-Actions.sql	Step 3: generate the three SQL scenarios.
04-Specific-Actions.kql	Step 4: find and interpret their actual audit records.

Record your environment values

Value	Your environment
Subscription ID / resource group	_____
SQL logical server / user database	_____
Workspace resource ID	_____

Use your own values throughout. SQL server names in script parameters and KQL exclude .database.windows.net. Database names must match exactly. Record event times in UTC.

1. Expand the audit policy

Use Azure Cloud Shell in the portal. The ordinary SQL Auditing blade does not expose custom audit-action-group selection.

1. Open https://portal.azure.com and sign in to your tenant. Select Cloud Shell (>_) in the toolbar > PowerShell. If prompted, use an approved setup; a no-storage session avoids creating a persistent mount. The scripts need Az.Accounts and Az.Sql, available in standard Cloud Shell.

2. Cloud Shell > Manage files / Upload: upload 01-Expand-Audit.ps1 and 02-Verify-Expanded-Audit.ps1. Use Get-ChildItem to confirm both are in the current directory.

3. Replace every value below with YOUR environment. To obtain the workspace resource ID: open the workspace > Overview > JSON View and copy id, or use Properties if Resource ID is displayed. A workspace can be in a different subscription if your permissions and network design permit it.

```
$subscription = '<YOUR-SUBSCRIPTION-GUID>'
$rg = '<YOUR-SQL-RESOURCE-GROUP>'
$server = '<YOUR-LOGICAL-SERVER-NAME>'
$workspace = '<YOUR-FULL-WORKSPACE-RESOURCE-ID>'
$demo = @{
  SubscriptionId = $subscription
  ResourceGroupName = $rg
  ServerName = $server
  WorkspaceResourceId = $workspace
}
```

Workspace resource ID format (one continuous value):

```
/subscriptions/<GUID>/resourceGroups/<RG>/providers/
Microsoft.OperationalInsights/workspaces/<WORKSPACE>
```

The line break above is for display only. Paste the complete resource ID as one string into $workspace.

4. Preview. Check the target and proposed groups; retain the printed original group list if you intend to restore the policy afterwards.

```
.\01-Expand-Audit.ps1 @demo -WhatIf
```

5. Apply, then approve the confirmation only for the intended disposable SQL server.

```
.\01-Expand-Audit.ps1 @demo
```

Required group	Scenario / purpose
SCHEMA_OBJECT_CHANGE_GROUP	Table CREATE / ALTER / DROP.
SCHEMA_OBJECT_ACCESS_GROUP	Access events including DELETE; also other object access.
DATABASE_PRINCIPAL_CHANGE_GROUP	Database-user CREATE / ALTER / DROP.
BATCH_COMPLETED_GROUP	Full completed-batch text, including removal statements.

Expected result Update completed, or all required groups already configured. Existing groups are retained; no diagnostic route is replaced. The broad access group and completed batches intentionally produce overlapping records. The script refuses multi-destination auditing or a nonempty predicate: review the configuration instead of bypassing the guard.

2. Verify the expanded audit

1. In the SAME Cloud Shell session, with $demo still defined, run:

```
.\02-Verify-Expanded-Audit.ps1 @demo
```

2. Require all four required groups to show Configured=True, the Log Analytics destination to match your workspace, and an enabled SQLSecurityAuditEvents diagnostic route on master. Existing additional groups are allowed.

Configuration gate Do not continue if verification fails. A successful read verifies configuration only; actual SQL event delivery is established in step 4. Do not resave the portal Auditing blade after expansion without re-verifying the groups.

Prove baseline delivery before generating the scenarios

3. Open your USER database > Query editor > Microsoft Entra authentication > Connect. Run this read-only marker and note GeneratedAtUtc. Alternatively use SSMS/VS Code.

```
SELECT N'SpecificAudit-Baseline' AS Marker,
       DB_NAME() AS DatabaseName,
       SYSUTCDATETIME() AS GeneratedAtUtc;
```

4. Open the selected Log Analytics workspace > Logs > KQL mode. Run the following query and locate YOUR fresh marker. Check the database/server fields in a raw result if the workspace receives logs from multiple sources.

```
AzureDiagnostics
| where TimeGenerated > ago(24h)
| where Category == "SQLSecurityAuditEvents"
| where tostring(column_ifexists("statement_s", ""))
    contains "SpecificAudit-Baseline"
| project TimeGenerated,
    Database=tostring(column_ifexists("database_name_s", "")),
    Statement=tostring(column_ifexists("statement_s", ""))
```

Allow ingestion A newly created diagnostic route can take up to 90 minutes to start delivery; established resource logs usually arrive within 3-10 minutes. Retry without repeatedly toggling/resaving the configuration. Old rows or another database's marker are not proof that your current path works.

If this gate fails

Check: the actual destination workspace and time range; audit enabled; required groups still present; SQLSecurityAuditEvents diagnostic route on master; fresh SQL successfully executed; workspace ingestion/quota settings; approved source-to-destination network access. An empty table alone does not identify the root cause.

No storage-role assignment is required for this Log Analytics-only exercise. A Contributor restriction on Add role assignment blocks managed-identity blob authorization, not automatically SQL audit export to Log Analytics. Separate SQL database permissions remain necessary.

3. Execute the database actions

Recommended: connect through SSMS or VS Code as the disposable database administrator. Open 03-Execute-Specific-Actions.sql, replace @ExpectedDatabase with YOUR demo database, and run the whole file while preserving its GO batches. Record RunStartUtc and RunEndUtc from the results.

Portal-only alternative: database > Query editor > Entra > Connect. Run the preflight below, then each scenario on the next page as separate code boxes with Run. Do not paste GO separators into these boxes.

Preflight - required for the portal alternative

```
SELECT DB_NAME() AS DatabaseName, @@TRANCOUNT AS OpenTransactions,
  OBJECT_ID(N'dbo.ExpAudit_TableLifecycle') AS ExistingLifecycle,
  OBJECT_ID(N'dbo.ExpAudit_DataRemoval') AS ExistingRemoval,
  DATABASE_PRINCIPAL_ID(N'ExpAudit_User') AS ExistingUser,
  SYSUTCDATETIME() AS RunStartUtc;
```

Proceed only if DatabaseName is YOUR disposable user database, never master; OpenTransactions=0; all three Existing... IDs are NULL. Stop if an object already exists. Do not drop an unknown object to force the demo to run.

A. Table CREATE / ALTER / DROP

Execute the following three boxes separately. If you ran the full supplied SQL file, do not repeat these statements.

```
CREATE TABLE dbo.ExpAudit_TableLifecycle
  (Id int NOT NULL PRIMARY KEY);
```

```
ALTER TABLE dbo.ExpAudit_TableLifecycle
  ADD Description nvarchar(50) NULL;
```

```
DROP TABLE dbo.ExpAudit_TableLifecycle;
```

SQL completion criteria for all three scenarios

Scenario	Expected SQL result
A - Table lifecycle	Table created, altered and dropped; final object ID NULL.
B - Data removal	Three seeded rows; DELETE leaves two; TRUNCATE leaves zero.
C - User lifecycle	Non-login user created/altered/dropped; final principal ID NULL.

The workload creates only its dedicated ExpAudit objects and does not require sample customer data. The non-login user needs no password, external directory lookup or real authentication. If a run stops halfway, inspect its objects before explicitly cleaning up only those names.

3. Actions continued - removal and users

For portal execution, run each code box separately in the intended USER database. Skip this page's execution if you already ran the full supplied SQL file.

B. DELETE versus TRUNCATE

```
CREATE TABLE dbo.ExpAudit_DataRemoval
  (Id int NOT NULL PRIMARY KEY, Label nvarchar(30) NOT NULL);
```

```
INSERT dbo.ExpAudit_DataRemoval (Id, Label)
VALUES (1,N'Keep'),(2,N'Delete'),(3,N'Truncate');
```

```
DELETE FROM dbo.ExpAudit_DataRemoval WHERE Id = 2;
```

```
SELECT COUNT(*) AS RemainingRows FROM dbo.ExpAudit_DataRemoval;
```

Expected RemainingRows=2. Continue with separate runs:

```
TRUNCATE TABLE dbo.ExpAudit_DataRemoval;
```

```
SELECT COUNT(*) AS RemainingRows FROM dbo.ExpAudit_DataRemoval;
```

Expected result RemainingRows=0. Leave the empty table until after the KQL checks. Native access records can reflect a permission check, so do not infer the number of deleted rows only from their affected_rows field.

C. Database-user CREATE / ALTER / DROP

```
CREATE USER ExpAudit_User WITHOUT LOGIN;
```

```
ALTER USER ExpAudit_User WITH DEFAULT_SCHEMA = dbo;
```

```
SELECT name, type_desc, default_schema_name
FROM sys.database_principals WHERE name = N'ExpAudit_User';
```

Expected: the user exists with default schema dbo. Then run:

```
DROP USER ExpAudit_User;
```

```
SELECT DATABASE_PRINCIPAL_ID(N'ExpAudit_User') AS PrincipalId,
       SYSUTCDATETIME() AS RunEndUtc;
```

Proceed to step 4 PrincipalId must be NULL after DROP. Record RunEndUtc. All three producers have now run. Allow their audit records to be ingested before evaluating the corresponding KQL results.

4. Find and interpret the audit records

1. Open YOUR Log Analytics workspace > Logs. Close any welcome/query overlay and switch to KQL mode.

2. Open 04-Specific-Actions.kql in a text editor. In EVERY block replace the database, logical server and UTC start time with YOUR run's values. Choose a start slightly before the first action. The placeholders below must be replaced before executing.

```
let TargetDatabase = "<your-demo-database>";
let TargetServer = "<your-sql-server>";
let RunStartUtc = datetime(<run-start-utc>);
```

For example, the timestamp must have the real form YYYY-MM-DDTHH:MM:SSZ. SQL UTC output supplies the date/time; do not paste the literal placeholder or your local time.

3. Copy ONE WHOLE block, including its let statements, into Logs and click Run. Replace the editor contents for the next query; do not append complete blocks accidentally.

KQL block	What must be visible
0 - Inventory	Actual action IDs/names and fresh event times for your selected server/database.
1 - Table lifecycle	Native CR / AL / DR referring to ExpAudit_TableLifecycle. BCM batch rows are labelled separately.
2 - Removal	Actual DELETE and TRUNCATE statements. Interpret native IDs AND statement text; completed batches remain separate.
3 - User lifecycle	Native CR / AL / DR referring to ExpAudit_User. These are principal changes, not sign-ins.

Important interpretation An engine can emit DL for both DELETE and TRUNCATE. Block 2 reads the actual statement to distinguish them; do not assume every DL is a DELETE statement. Native behavior can vary. If you see only BCM, you have not yet proved granular capture.

The new KQL trims all action-ID padding using trim(@"\s+", ...). Overlapping granular/batch records are expected; event counts are not counts of distinct SQL statements. A succeeded flag can describe a permission check, not proof that every statement committed.

After the demonstration

4. Once all required records have been observed, remove only the empty dedicated table:

```
DROP TABLE dbo.ExpAudit_DataRemoval;
```

5. The successful workload already dropped the lifecycle table and non-login user. If restoring the original audit groups, use the list printed in step 1 and review any intervening administrator changes. Do not blindly overwrite another person's policy changes. Review ingestion costs before retaining broad groups.

If records are missing

Check the matching producer actually ran; the groups and master route persisted; the server/database/UTC values are correct; the actual workspace is selected; delivery latency and network/ingestion controls permit the path. Do not lower thresholds, fabricate events or label batch text as native proof.

References: Microsoft Learn - Set-AzSqlServerAudit; SQL Server Audit action groups and actions; Diagnostic settings (expected initial latency); Azure Monitor network security perimeter.

https://learn.microsoft.com/powershell/module/az.sql/set-azsqlserveraudit

https://learn.microsoft.com/azure/azure-monitor/data-collection/diagnostic-settings#expected-latency-after-creating-a-diagnostic-setting